Enabling Google Cloud Borderless Lakehouse with Azure Databricks Unity Catalog

This guide provides step-by-step instructions to connect **Google Cloud Borderless Lakehouse (BigLake Iceberg REST Catalog Federation)** directly to **Azure Databricks Unity Catalog** (`*.azuredatabricks.net` backed by Azure Data Lake Storage Gen2 `abfss://`), enabling zero-copy SQL queries from **BigQuery** and **Vertex AI / ADK Agents** in `europe-west2` (London).

1. Architectural Overview: Open Sharing (`config.share`) vs. Borderless Lakehouse Federation

Dimension	Current Setup: Databricks Open Sharing (`config.share`)	Target Setup: Borderless Lakehouse Federation (`--federated-catalog-type=unity`)
Protocol & Endpoint	Delta Sharing REST protocol (`/api/2.0/opensharing/...`)	Apache Iceberg REST Catalog v1 API (`/api/2.1/unity-catalog/iceberg-rest/v1`)
Catalog Authentication	Pre-issued static `bearerToken` in `.share` files	**Single Databricks Service Principal** via **Secretless GCP OIDC Federation** (recommended) or **OAuth M2M (`client_id` / `client_secret`)** in Regional Secret Manager
Supported Source Objects	Delta Tables and Dynamic Views	**Physical Delta Tables in an External Location** with **Delta UniForm (`IcebergCompatV2`)** enabled *(SQL Views and tables with `deletionVectors` or Unity Catalog Row Filters/Masks are not exposed by the Iceberg REST API)*
Storage Access	Vends short-lived pre-signed ADLS URLs per file	Vends short-lived scoped ADLS SAS tokens (`adls.sas-token.*`) via `X-Iceberg-Access-Delegation: vended-credentials`
BigQuery Experience	Hydrates Parquet snapshots into native BigQuery tables	**Zero-copy query in-place** using 4-part table names (`project.catalog.namespace.table`) with automatic local block caching in GCP (`europe-west2`)
End-User Row-Level Security (RLS)	Native BigQuery `ROW ACCESS POLICY` + `v_bookings` view via Workforce Identity Federation (`SESSION_USER()`)	BigQuery `v_bookings` RLS view over the 4-part federated table via Workforce Identity Federation (`SESSION_USER()`)

2. Phase 1: Azure Databricks Prerequisites & Table Preparation

Complete the following 5 steps in the Azure Databricks workspace (`adb-3349553429796990.10.azuredatabricks.net`) and Azure Portal:

Step 1.1: Enable External Data Access on the Unity Catalog Metastore

By default, Unity Catalog disables credential vending to external Iceberg engines.

1. Log in to the **Azure Databricks Account Console** (or Workspace Admin Settings).
2. Navigate to **Catalog** → select the target **Metastore** (`cc4946a1-1075-47ac-aa46-6a34352cb3db`) → **Access** tab.
3. Enable **External data access** (*Allow external engines to access data in this metastore via credential vending*).

Step 1.2: Enable Delta UniForm (`IcebergCompatV2`) & Materialize Shared Views

Unity Catalog's Iceberg REST endpoint (`/api/2.1/unity-catalog/iceberg-rest/v1`) enforces three requirements on shared tables:

1. **Physical Tables Only (No SQL Views):** Dynamic SQL `VIEW` s are not exposed over the Iceberg REST API. Any view (such as `bookings`) must be materialized as a physical Delta table stored in an ADLS Gen2 External Location.

2. **Valid Column Aliases for Masked Expressions:** Calculated expressions such as `concat(substring(AgentName, 1, 2), '******')` must have explicit SQL aliases (`AS AgentName` and `AS LeadPassenger`) so they map to valid column names in Iceberg and BigQuery.
3. **Disable Deletion Vectors & Do Not Attach Unity Catalog Row Filters/Column Masks:**
   - Delta UniForm (`IcebergCompatV2`) requires `'delta.enableDeletionVectors' = 'false'` .
   - Do **not** attach Unity Catalog `ROW FILTER` or `COLUMN MASK` policies onto the federated tables, as Unity Catalog blocks Iceberg storage credential vending when dynamic row filters or masks are present. Static masking should be baked into the `SELECT` query below, and dynamic per-user Row-Level Security is enforced in BigQuery via `SESSION_USER()` .

A. For `bookings` (Materializing the View into a Physical Delta UniForm Table)

```
CREATE OR REPLACE TABLE <unity_catalog_name>.google_sharing.bookings_uniform
USING DELTA
LOCATION 'abfss://<container>@dlsdlhdevuksouth001.dfs.core.windows.net/google_sharing/bookings_uniform'
TBLPROPERTIES (
  'delta.enableDeletionVectors' = 'false',
  'delta.columnMapping.mode' = 'name',
  'delta.enableIcebergCompatV2' = 'true',
  'delta.universalFormat.enabledFormats' = 'iceberg'
)
AS SELECT
  BookingReference,
  DeltaId,
  LastModified,
  ABTA,
  ATOL,
  Adults,
  -- Alias masked columns to their standard schema names:
  concat(substring(AgentName, 1, 2), '******') AS AgentName,
  BalanceDue,
  BondingType,
  BookingDate,
  BookingSource,
  Branch,
  Cancelled,
  Children,
  Commission,
  CreatedDate,
  DebtorID,
  DepartureDate,
  Duration,
  GrossPrice,
  Infants,
  IsActiveDirectDebit,
  concat(substring(LeadPassenger, 1, 2), '******') AS LeadPassenger,
  Outstanding,
  Passengers,
  Profit,
  ReturnDate,
  SaleBranchID,
  TotalPayment,
  Username,
  AE_SupplierKey,
  DestinationType,
  DeparturePoint,
  ArrivalPoint,
  FlightDetails,
  HotelDetails,
  CruiseDetails,
  BuyingBranch,
  SellingBranch,
  CarParkDetails,
  InsuranceDetails,
  AttractionsEventsDetails
  -- (Include remaining columns from the source view/table)
FROM <source_catalog>.<source_schema>.<source_bookings_table_or_view>;
```

B. For Existing Base Tables (`customermaster` , `customermosaicdemographics` , `supplier` , `branch`)

If enabling UniForm in-place on an existing physical Delta table that currently has `deletionVectors` enabled (`minReaderVersion: 3`), disable deletion vectors, purge existing deletion vectors with `REORG TABLE` , and enable `IcebergCompatV2` :

```
-- Repeat for customermaster, customermosaicdemographics, supplier, and branch:
ALTER TABLE <unity_catalog_name>.google_sharing.customermaster
SET TBLPROPERTIES (
  'delta.enableDeletionVectors' = 'false',
  'delta.columnMapping.mode' = 'name'
);

REORG TABLE <unity_catalog_name>.google_sharing.customermaster APPLY (PURGE);

ALTER TABLE <unity_catalog_name>.google_sharing.customermaster
SET TBLPROPERTIES (
  'delta.enableIcebergCompatV2' = 'true',
  'delta.universalFormat.enabledFormats' = 'iceberg'
);
```

Step 1.3: Create a Databricks Service Principal & Grant Privileges

1. In Azure Databricks, go to **Settings → Identity and access → Service principals → Add service principal**.
2. Create a new Databricks-managed Service Principal (for example, `gcp-biglake-federation`) and copy its **Application ID (UUID)**.
3. Grant the Service Principal read access and the mandatory `EXTERNAL USE SCHEMA` privilege (which allows Unity Catalog to vend short-lived ADLS SAS tokens to BigQuery):

```
GRANT USE CATALOG
  ON CATALOG <unity_catalog_name>
  TO `<SERVICE_PRINCIPAL_APPLICATION_ID>`;

GRANT USE SCHEMA, EXTERNAL USE SCHEMA, SELECT
  ON SCHEMA <unity_catalog_name>.google_sharing
  TO `<SERVICE_PRINCIPAL_APPLICATION_ID>`;
```

Step 1.4: Verify Azure ADLS Gen2 Storage Firewall Rules

Because BigQuery workers in `europe-west2` read the Parquet files directly from `dlsdlhdevuksouth001.dfs.core.windows.net` using the SAS tokens vended by Unity Catalog, verify in the **Azure Portal** (**Storage accounts** → `dlsdlhdevuksouth001` → **Networking** → **Firewalls and virtual networks**) that external read requests authenticated with valid SAS tokens from Google Cloud (`europe-west2`) are permitted.

Step 1.5: Parameters to Share with the Google Cloud Team

Provide the following 3 values to the Google Cloud team to complete the catalog link:

1. **Azure Databricks Workspace Hostname:** `adb-3349553429796990.10.azuredatabricks.net` *(must be the standard workspace URL ending in* `.azuredatabricks.net` *, not the* `*.opensharing.uksouth.azuredatabricks.net` *endpoint)*
2. **Unity Catalog Name & Schema Name:** `<unity_catalog_name>` and `google_sharing`
3. **Service Principal Application ID (UUID):** `<SERVICE_PRINCIPAL_APPLICATION_ID>` *(or an OAuth* `client_id` *and* `client_secret` *if using Option B below)*

---

3. Phase 2: Google Cloud Setup (`europe-west2`)

Run the following commands in the target Google Cloud project (e.g., `PROJECT_ID="haystravel-agentic26lon-281"`).

Step 2.1: Enable Required APIs

```
export PROJECT_ID="haystravel-agentic26lon-281"

gcloud services enable \
  biglake.googleapis.com \
  bigquery.googleapis.com \
  secretmanager.googleapis.com \
  --project="${PROJECT_ID}"
```

Step 2.2: Create the Federated Lakehouse Catalog

Choose **Option A (Secretless OIDC Federation — Recommended)** or **Option B (Regional Secret Manager — OAuth Client Secret)**:

Option A: Secretless OIDC Federation (Recommended)

```
# 1. Create the federated catalog in europe-west2 with cross-cloud caching enabled
gcloud alpha biglake iceberg catalogs create hays_unity_catalog \
  --project="${PROJECT_ID}" \
  --primary-location="europe-west2" \
  --catalog-type="federated" \
  --federated-catalog-type="unity" \
  --unity-instance-name="adb-3349553429796990.10.azuredatabricks.net" \
  --unity-catalog-name="<UNITY_CATALOG_NAME>" \
  --unity-service-principal-application-id="<SERVICE_PRINCIPAL_APPLICATION_ID>" \
  --cross-cloud-cache="enabled" \
  --refresh-interval="300s" \
  --namespace-filters="google_sharing"

# 2. Retrieve the 21-digit numeric Google Service Account Unique ID
gcloud alpha biglake iceberg catalogs describe hays_unity_catalog \
  --project="${PROJECT_ID}" \
  --format='value(biglake-service-account-id)'
```

(Share the 21-digit `biglake-service-account-id` output from command #2 with the Databricks administrator for Phase 3).

Option B: Secret-Based Authentication (Regional Secret Manager)

If using a Databricks Service Principal OAuth secret (`client_id` and `client_secret`) instead of OIDC federation:

```
# 1. Point gcloud to the regional Secret Manager endpoint for europe-west2
gcloud config set api_endpoint_overrides/secretmanager \
  https://secretmanager.europe-west2.rep.googleapis.com/

# 2. Create a regional secret containing {"client_id": "...", "client_secret": "..."}
gcloud secrets create databricks-unity-oauth \
  --location="europe-west2" \
  --project="${PROJECT_ID}" \
  --data-file=credentials.json

# 3. Create the federated catalog referencing the regional secret
gcloud alpha biglake iceberg catalogs create hays_unity_catalog \
  --project="${PROJECT_ID}" \
  --primary-location="europe-west2" \
  --catalog-type="federated" \
  --federated-catalog-type="unity" \
  --unity-instance-name="adb-3349553429796990.10.azuredatabricks.net" \
  --unity-catalog-name="<UNITY_CATALOG_NAME>" \
  --secret-name="projects/${PROJECT_ID}/locations/europe-west2/secrets/databricks-unity-oauth" \
  --cross-cloud-cache="enabled" \
  --refresh-interval="300s" \
  --namespace-filters="google_sharing"

# 4. Grant the catalog's provisioned BigLake service account access to the secret
gcloud secrets add-iam-policy-binding databricks-unity-oauth \
  --project="${PROJECT_ID}" \
  --location="europe-west2" \
  --member="serviceAccount:$(gcloud alpha biglake iceberg catalogs describe hays_unity_catalog \
    --project="${PROJECT_ID}" \
    --format='value(biglake-service-account)')" \
  --role="roles/secretmanager.secretAccessor"
```

4. Phase 3: Complete OIDC Trust Handshake in Azure Databricks (Option A Only)

If using **Option A (Secretless OIDC Federation)**, add the trust policy on the Databricks Service Principal:

1. In Azure Databricks, open **Settings** → **Identity and access** → **Service principals** → select the Service Principal created in Step 1.3.
2. Open the **Federation policies** tab → click **Add policy**.
3. Enter the following exact values:
   - **Issuer URL:** `https://accounts.google.com`
   - **Subject:** `<biglake-service-account-id>` *(the 21-digit numeric Google Service Account ID from Step 2.2)*
   - **Audience:** `https://accounts.cloud.databricks.com`
4. Click **Save**.

5. Phase 4: Verify Synchronization, Querying & Caching in BigQuery

Step 4.1: Verify Catalog & Table Synchronization

BigLake automatically syncs namespaces and Iceberg table metadata from Unity Catalog every 5 minutes (`--refresh-interval="300s"`).

```
# Check catalog refresh status
gcloud alpha biglake iceberg catalogs describe hays_unity_catalog \
  --project="${PROJECT_ID}"

# Confirm that the google_sharing namespace and tables have synchronized
gcloud alpha biglake iceberg namespaces list \
  --project="${PROJECT_ID}" \
  --catalog="hays_unity_catalog"

gcloud alpha biglake iceberg tables list \
  --project="${PROJECT_ID}" \
  --catalog="hays_unity_catalog" \
  --namespace="google_sharing"
```

Step 4.2: Query Directly in BigQuery (4-Part `P.C.N.T` Syntax)

Query the federated catalog directly in BigQuery Studio or via the `bq` CLI using `project.federated_catalog.namespace.table`:

```
SELECT
  BookingReference,
  SaleBranchID,
  SellingBranch.BranchName AS BranchName,
  AgentName,
  BookingDate,
  GrossPrice,
  Commission
FROM `haystravel-agentic26lon-281.hays_unity_catalog.google_sharing.bookings_uniform`
WHERE BookingDate >= '2026-10-01'
  AND Cancelled = 'N'
LIMIT 20;
```

Step 4.3: Verify Intelligent Cross-Cloud Caching (Minimizing Azure Egress)

When BigQuery reads Parquet blocks from Azure ADLS (`uksouth`), BigLake automatically caches those blocks locally in `europe-west2` (`--cross-cloud-cache="enabled"`). Subsequent queries read from the local GCP cache rather than re-fetching from Azure.

Verify cache hit statistics on any BigQuery job ID:

```
bq show --format=prettyjson --j <JOB_ID> | grep -A 6 "objectStorageStats"
```

- `cloudProvider`: `"AZURE"`
- `objectStorageBytesRead`: Bytes fetched over the network from Azure ADLS Gen2
- `cacheBytesRead`: Bytes served directly from the local `europe-west2` cache

---

6. End-User Row-Level Security (RLS) with Workforce Identity Federation (WIF) & OIDC

Because 4-part Federated Catalog tables (`project.hays_unity_catalog.google_sharing.bookings_uniform`) do not support base-table `CREATE ROW ACCESS POLICY` or Databricks-side `ROW FILTER`s, **4-tier Row-Level Security (`BRANCH_STAFF`, `BRANCH_MANAGER`, `REGIONAL_MANAGER`, `DIVISIONAL_HEAD`) is enforced in BigQuery using Workforce Identity Federation (WIF) + OIDC (`SESSION_USER()`) and the `v_bookings` view**:

Step 6.1: How Identity Flows from Microsoft Entra ID to BigQuery

1. Colleagues sign in with **Microsoft Entra ID** and receive an OIDC ID Token (`id_token` JWT).
2. The ADK Agent exchanges that OIDC JWT at Google Cloud's Security Token Service (`https://sts.googleapis.com/v1/token`) against the GCP Workforce Identity Pool (`hays-oidc-pool`)—**no custom per-agent service accounts are required**.
3. BigQuery executes all agent tool queries under the colleague's federated identity (`SESSION_USER() = 'principal://iam.googleapis.com/locations/global/workforcePools/hays-oidc-pool/subject/<user_upn>'`).

Step 6.2: Point the `v_bookings` RLS View to the 4-Part Federated Table

Create or update the `v_bookings` view in BigQuery so it joins the 4-part federated `bookings_uniform` table with `user_entitlements` on `SESSION_USER()`:

```
CREATE OR REPLACE VIEW `haystravel-agentic26lon-281.hays_travel_sharing.v_bookings` AS
SELECT b.*
FROM `haystravel-agentic26lon-281.hays_unity_catalog.google_sharing.bookings_uniform` b
INNER JOIN `haystravel-agentic26lon-281.hays_travel_sharing.user_entitlements` e
  ON (e.wif_subject = SESSION_USER() OR e.user_email = SESSION_USER())
WHERE
  (e.role = 'BRANCH_STAFF' AND b.SaleBranchID = e.branch_id AND b.Username = e.username)
  OR (e.role = 'BRANCH_MANAGER' AND b.SaleBranchID = e.branch_id)
  OR (e.role = 'REGIONAL_MANAGER' AND b.SellingBranch.BranchGroup = e.region_group)
  OR (e.role = 'DIVISIONAL_HEAD' AND (e.division = 'ALL' OR b.SellingBranch.BranchBusinessDivision =
e.division));
```